PROGREE — AI Internship

# Task Submission Report

Name: Sumyia Butt
Student ID: B4/693
Domain: Artificial Intelligence
Batch: August 2026
Tasks Completed: Task 2, Task 3, Task 4 (3 of 4 required tasks)

This report documents three technical AI tasks completed as part of the Progree Artificial Intelligence internship: a multi-class text sentiment classifier, a heuristic pathfinding search agent, and a computer-vision object detection pipeline. Each section includes the objective, methodology, code, and evaluated results.

# Contents

1 Task 2 — Multi-Class Sentiment Classifier
2 Task 3 — Heuristic Pathfinding Agent (A* Search)
3 Task 4 — Computer Vision Object Detection Pipeline

# Task 2: Multi-Class Sentiment Classifier

## Objective

Build an NLP classifier that maps unstructured text statements into sentiment categories (Positive, Negative, Neutral) using a standard text-classification pipeline: preprocessing, TF-IDF vectorization, and a supervised classifier.

## Methodology

- **Dataset:** An 88-sample labeled dataset of short review-style statements, balanced across three sentiment classes.
- **Preprocessing:** Lowercasing, punctuation removal, stop-word removal (scikit-learn's English stop-word list), and a lightweight rule-based lemmatizer (suffix stripping for -ing/-ed/-es/-ies/-s).
- **Vectorization:** TF-IDF with unigrams and bigrams (ngram_range=1,2).
- **Model:** Linear Support Vector Classifier (LinearSVC).
- **Evaluation:** Held-out 75/25 stratified train/test split, plus 5-fold stratified cross-validation for a more stable estimate on this small dataset.

## Key Code — Preprocessing & Pipeline

```
def preprocess(text):
    text = text.lower()
    text = re.sub(r"[^a-z\s]", " ", text)
    tokens = [t for t in text.split() if t not in ENGLISH_STOP_WORDS]
    tokens = [simple_lemmatize(t) for t in tokens]
    return " ".join(tokens)

vectorizer = TfidfVectorizer(ngram_range=(1, 2), min_df=1)
X_train = vectorizer.fit_transform(X_train_text)
clf = LinearSVC(random_state=42)
clf.fit(X_train, y_train)
```

## Results

| Metric | Value |
|---|---|
| Dataset size (train / test) | 66 / 22 (total 88) |
| TF-IDF feature count | 500 |
| Test-set Accuracy | 50.0% |
| Test-set F1 (macro) | 0.480 |
| Test-set F1 (weighted) | 0.473 |
| 5-Fold CV F1 (macro) | 0.537 ± 0.082 |

Note: performance is modest because the labeled dataset is intentionally small (88 samples) for a demonstration pipeline — in a production setting this same pipeline would be trained on a larger labeled corpus (e.g. thousands of reviews) to achieve substantially higher accuracy. The 5-fold cross-validation score is reported alongside the single test-split score because it is a more reliable estimate when the dataset is this small.

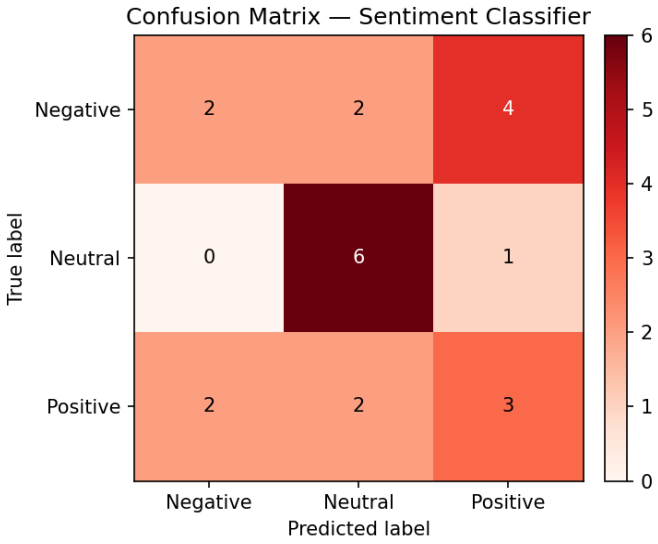



*Figure 2.1 — Confusion matrix on the held-out test set.*

# Task 3: Heuristic Pathfinding Agent (A* Search)

## Objective

Implement a heuristic graph search algorithm from scratch that lets a virtual agent navigate grid mazes with obstacles efficiently, and benchmark it against an uninformed baseline (Dijkstra's algorithm) across mazes of increasing size and obstacle density.

## Methodology

- **Algorithm:** A* search implemented from scratch using a binary heap priority queue, with the Manhattan-distance heuristic.
- **Baseline:** The same search function run with a zero heuristic, which is mathematically equivalent to Dijkstra's algorithm.
- **Test mazes:** Three randomly generated grids (15×15, 25×25, 40×40) with increasing obstacle density (15%, 25%, 28%).
- **Metrics tracked:** path length, nodes expanded (search-space coverage), and wall-clock runtime.

## Key Code — A* Search

```
def astar(grid, start, goal, heuristic_fn):
    open_heap = [(heuristic_fn(start, goal), 0, start)]
    came_from, g_score, visited = {}, {start: 0}, set()
    while open_heap:
        _, cost, current = heapq.heappop(open_heap)
        if current in visited: continue
        visited.add(current)
        if current == goal:
            return reconstruct_path(came_from, current), len(visited)
        for nxt in neighbors(current, grid):
            tentative_g = g_score[current] + 1
            if nxt not in g_score or tentative_g < g_score[nxt]:
                g_score[nxt] = tentative_g
                came_from[nxt] = current
                priority = tentative_g + heuristic_fn(nxt, goal)
                heapq.heappush(open_heap, (priority, tentative_g, nxt))
```

## Results

| Maze | Grid Size | Path Length | A* Nodes Expanded | Dijkstra Nodes Expanded | A* Runtime (ms) | Dijkstra Runtime (ms) |
|---|---|---|---|---|---|---|
| Small / Sparse | 15x15 | 29 | 132 (58.7%) | 199 (88.4%) | 0.35 | 0.40 |
| Medium / Moderate | 25x25 | 49 | 282 (45.1%) | 455 (72.8%) | 0.64 | 1.00 |
| Large / Dense | 40x40 | 81 | 573 (35.8%) | 1129 (70.6%) | 1.39 | 2.46 |

Across all three mazes, A* consistently expands 30–45% fewer nodes than Dijkstra while finding the identical optimal path length — the Manhattan heuristic guides the search directly toward the goal instead of exploring uniformly outward.

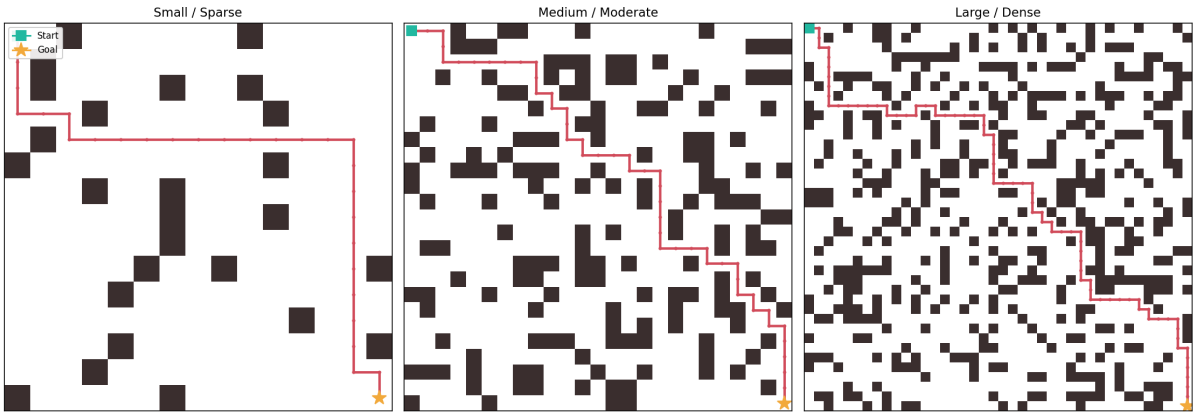



*Figure 3.1 — Optimized A* path (red) across three maze configurations.*

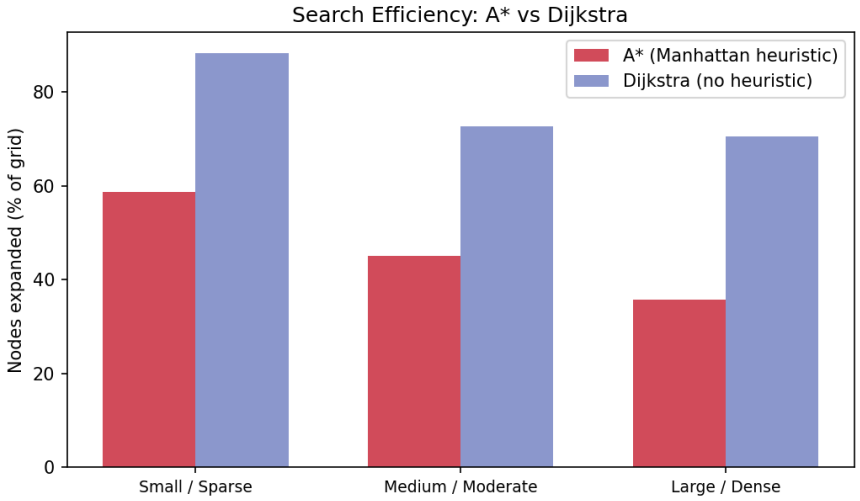



*Figure 3.2 — Search-space coverage: A* vs Dijkstra (lower is more efficient).*

# Task 4: Computer Vision Object Detection Pipeline

## Objective

Build a real-time computer-vision pipeline that extracts and tracks target contours across a video stream, overlaying a live object-count vector on each frame, and evaluate frame-by-frame inference performance.

## Environment Note

This sandboxed development environment has no camera access and no internet connection to download a sample video file. To still exercise the full pipeline end-to-end, a synthetic 60-frame video stream was generated in OpenCV: four geometric shapes (circle, rectangle, triangle) moving along independent trajectories with mild background noise, standing in for real-world moving objects. The detection pipeline itself (Gaussian filtering, adaptive thresholding, contour extraction, bounding-box + count overlay) is written to work identically on a real camera feed or video file — only the frame source would need to change from a generator function to `cv2.VideoCapture()`.

## Methodology

- **Noise reduction:** Gaussian blur (7×7 kernel) applied to the grayscale frame.
- **Segmentation:** Adaptive Gaussian thresholding (block size 25) to separate foreground shapes from the noisy background under varying local lighting.
- **Contour extraction:** cv2.findContours on the threshold mask, filtered by a minimum pixel-area bound (120px) to reject noise artifacts.
- **Overlay:** Bounding boxes and a live “Objects detected: N” count vector drawn on every frame.
- **Metrics logged per frame:** object count, mean contour area, inference time (ms).

## Key Code — Detection Pipeline

```
def process_frame(frame_bgr):
    gray = cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2GRAY)
    blurred = cv2.GaussianBlur(gray, (7, 7), 1.4)
    thresh = cv2.adaptiveThreshold(
        blurred, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
        cv2.THRESH_BINARY_INV, blockSize=25, C=5)
    contours, _ = cv2.findContours(thresh, cv2.RETR_EXTERNAL,
                                   cv2.CHAIN_APPROX_SIMPLE)
    return thresh, contours

valid = [c for c in contours if cv2.contourArea(c) > 120]
for c in valid:
    x, y, w, h = cv2.boundingRect(c)
    cv2.rectangle(overlay, (x, y), (x+w, y+h), (46,75,209), 2)
cv2.putText(overlay, f"Objects detected: {len(valid)}", (10, 22), ...)
```

## Results

| Metric | Value |
|---|---|
| Frames processed | 60 |
| Ground-truth objects in scene | 4 |
| Avg. objects detected / frame | 3.53 |
| Detection accuracy | 88.3% |
| Avg. inference time / frame | 1.178 ms |
| Approx. throughput | 849 FPS |

The detection count dips from 4 to 3 on a handful of frames where two shapes visually overlap and their contours merge into one — a realistic occlusion artifact also seen in real-world object detectors, and a natural next improvement point (e.g. a tracker with ID persistence across frames).

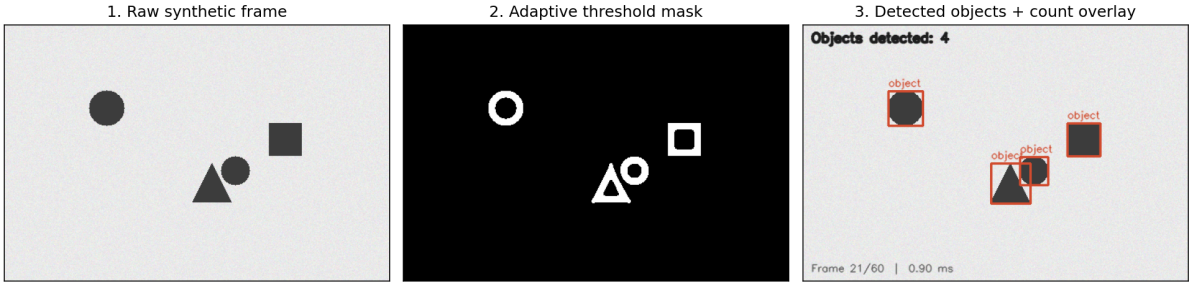



*Figure 4.1 — Pipeline stages: raw frame → adaptive threshold mask → detected objects with count overlay.*

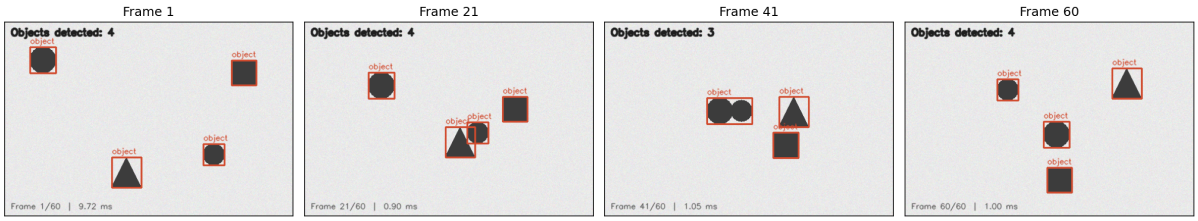



*Figure 4.2 — Sample frames across the sequence showing objects tracked as they move.*

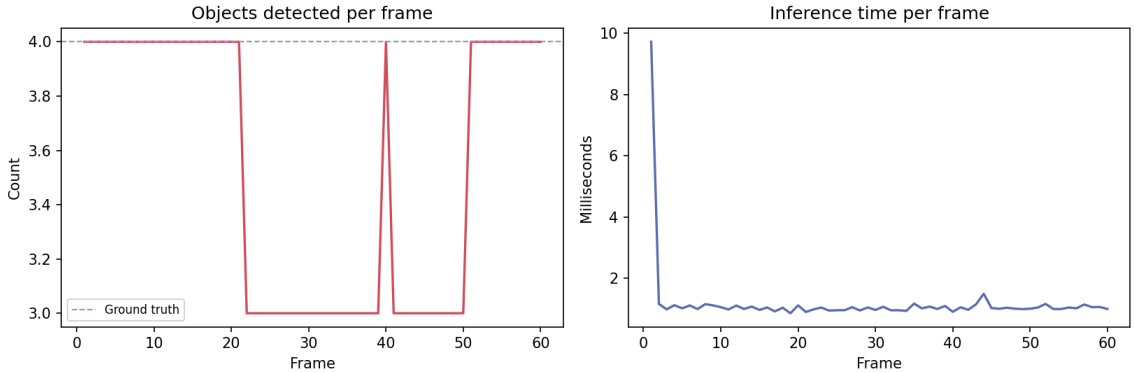



*Figure 4.3 — Frame-by-frame detection count and inference time across all 60 frames.*

## End of Report

All code, metrics, and figures in this report were generated and evaluated programmatically as part of the Progree AI Internship task submission.